Østfold University College

MPI Programming in Java

Outline

- MPI and MPI for Java (MPJ)
- APIs
- Communication Modes

MPI Programming

MPI: The Message Passing Interface

- MPI is an API for communication between processes
  - API for sending and receiving messages
- Low-level parts of API:
  - Fast data transfer, support multiple modes of message synchronizations
- High level parts of API:
  - Concerned with organization of process groups
  - Provide collective communications in parallel apps
- Widely used in parallel programming for high performance computing (HPC) due to its high portability, scalability, good performance

MPI: A program paradigm

- MPI supports SPMD model of parallel programming
  - Group of multiple processes in a program
  - Each process work on local data
  - Communications provided by a shared communicator instead of socket pairs
- Deployed in both shared-memory and distributed –memory systems

MPI: A program paradigm

- First, MPI supported communication among processes within a computer

- Extended to communication among processes in different computers
  - Needs collective operations, Remote-memory access operations, Dynamic process creation, Parallel I/O

MPI: A Specification

- Message-passing library interface specification
  - Defines API for communications in terms of syntax, semantics
  - For example:

- Not an implementation
  - Multiple implementations of MPI

MPJ

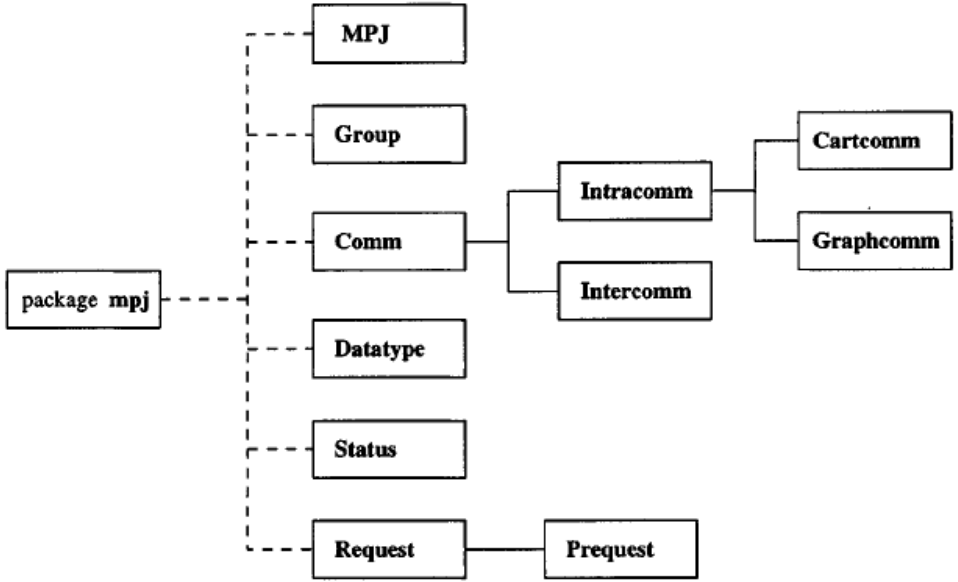


Figure 1. Principal classes of MPJ.

MPI: A Specification - Example

- Syntax of blocking send()

MPI_SEND(buf, count, datatype, dest, tag, comm)

IN	buf	initial address of send buffer (choice)
IN	count	number of elements in send buffer (non-negative integer)
IN	datatype	datatype of each send buffer element (handle)
IN	dest	rank of destination (integer)
IN	tag	message tag (integer)
IN	comm	communicator (handle)

```
int MPI_Send(const void* buf, int count, MPI_Datatype datatype, int dest,
             int tag, MPI_Comm comm)
```

MPI: A Specification - Example

- Semantics of blocking send()

`MPI_Send()` sends the exact `count` of elements,

...

The send call described in Section 3.2.1 is **blocking**: it does not return until the message data and envelope have been safely stored away so that the sender is free to modify the send buffer. The message might be copied directly into the matching receive buffer, or it might be copied into a temporary system buffer.

MPI in Java

- MPJ: MPI-like message passing for Java
- Explosion of MPI implementation for Java:
  - mpiJava, JMPI, jPVM, FastMPJ, MPJ Express, MPJ/Ibis
- MPJ: a specification by the Message-Passing Working Group of Java Grande Forum
  - MPJ: MPI-like message passing for Java, B. Carpenter, V. Getov, G. Judd, A. Skjellum, G. Fox, Concurrency Practice and Experience, 12(11), 2000

MPJ Express

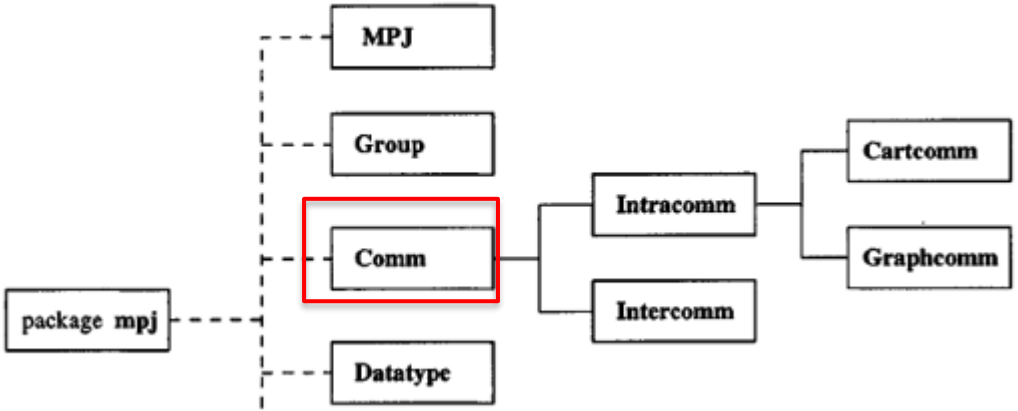
- Mature, easy-to-use
- Supports wide range of communication protocols, used in many platform
- Offers to configuration
  - Multicore configuration: shared-memory systems
  - Cluster configuration: distributed-memory system

Østfold University College

Communication modes

Point-to-Point communications

- Point-to-point communication: send and receive operations
- Methods in Comm class



Send() operation

- Blocking send

```
void Send(java.lang.Object buf,
                int offset,
                int count,
                Datatype datatype,
                int dest,
                int tag)
                throws MPIException
```

(`buf`: data buffer
`offset`: initial offeset in sent buffer
`count`: number of items to send
`datatype`: datatype of each item in send buffer
`dest`: rank of receiving process
`tag`: message tag)

Blocking send: sending process is blocked until the message is received or acknowledged by the receiving process

Example

```
array[0] = 42;
MPI.COMM_WORLD.Send(array, 0, 1, MPI.INT, 1, 8);
```

- sending the first element of the array to process rank 1

Send() operation

Blocking send

```
void Send(java.lang.Object buf,
          int offset,
          int count,
          Datatype datatype,
          int dest,
          int tag)
          throws MPIException
```

(`buf`: data buffer
`offset`: initial offeset in sent buffer
`count`: number of items to send
`datatype`: datatype of each item in send buffer
`dest`: rank of receiving process
`tag`: message tag)

Blocking send: sending process is blocked until the message is received or acknowledged by the receiving process

Non-blocking send

```
Request Isend(java.lang.Object buf,
        int offset,
        int count,
        Datatype datatype,
        int dest,
        int tag)
        throws MPIException
```

Non-blocking send: sending processes is allowed as soon as the message has been copied to a local buffer

Receive() operation

- Blocking receive

```
Status Recv(java.lang.Object buf,
        int offset,
        int count,
        Datatype datatype,
        int source,
        int tag)
        throws MPIException
```

```
if(me==0){
    data[0]=random.nextInt( bound: 100);
    MPI.COMM_WORLD.Send(data, offset: 0, count: 1,MPI.INT, dest: 1, tag: 10);
    System.out.println("Process "+me+" sends number "+data[0]+" to Process 1");
}
else {
    MPI.COMM_WORLD.Recv(data, offset: 0, count: 1,MPI.INT, source: 0, tag: 10);
    System.out.println("Process "+me+" receives number "+data[0]+" from Process 0");
}
```

Blocking receive: the receiving process is blocked until a message is available.

Receive() operation

- Blocking receive

```
Status Recv(java.lang.Object buf,
        int offset,
        int count,
        Datatype datatype,
        int source,
        int tag)
        throws MPIException
```

- The receive buffer consists of the storage containing count consecutive elements of the type specfied by datatype, starting at address buf.
- The length of the received message must be less than or equal to the length of the receive buffer. An oveflow error occurs if all incoming data does not fit, without truncation, into the receive buffer.
- If a message that is shorter than the receive buffer arrives, then only those locations corresponding to the (shorter) message are modified

Blocking receive: the receiving process is blocked until a message is available.

Receive() operation

- Blocking receive

```
Status Recv(java.lang.Object buf,
        int offset,
        int count,
        Datatype datatype,
        int source,
        int tag)
        throws MPIException
```

Blocking receive: the receiving process is blocked until a message is available.

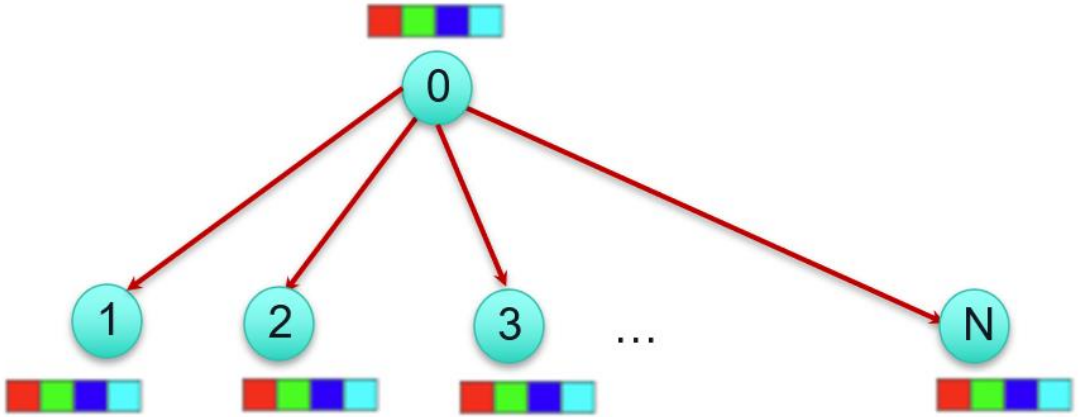
Non-blocking receive

```
Status Irecv(java.lang.Object buf,
        int offset,
        int count,
        Datatype datatype,
        int source,
        int tag)
    throws MPIException
```

Non-blocking receive: receiving process can continue executing even if no message is available.

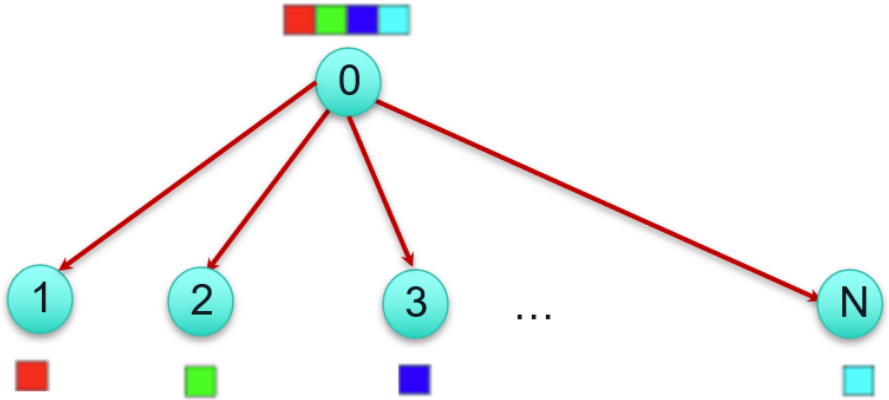
Collective communications - Broadcast

- The root process sends identical data to all other processes
- There are N copies of the original data – one for each process



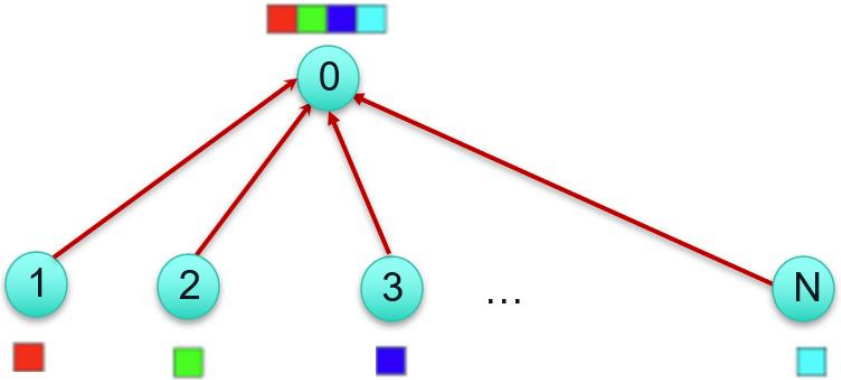
Collective communications - Scatter

- The root process starts with N unique messages
- Each message is destined to each process
- Does not involve any duplication of data



Collective communications - Gather

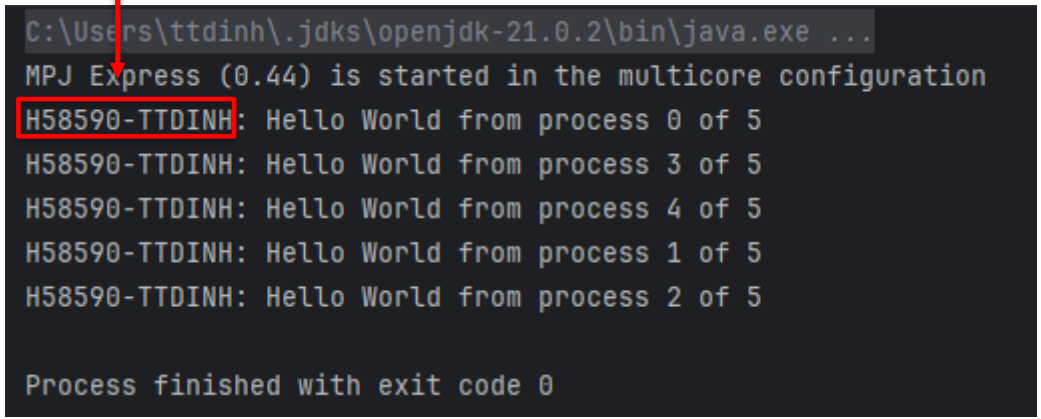
- The root process collects a unique message from each process
- Concatenation
- Does not evolve any combination or reduction of data



MPI APIs – Comm class

```
public void barrier() throws MPJException {...}

public void bcast(Object buffer, int offset, int count,
                  Datatype datatype, int root) throws MPJException {...}

public void gather(Object sendbuf, int sendoffset, int sendcount, Datatype sendtype,
                   Object recvbuf, int recvoffset, int recvcount, Datatype recvtype,
                   int root) throws MPJException {...}

public void gatherv(Object sendbuf, int sendoffset, int sendcount, Datatype sendtype,
                    Object recvbuf, int recvoffset, int [] recvcounts, int [] displs,
                    Datatype recvtype, int root) throws MPJException {...}

public void scatter(Object sendbuf, int sendoffset, int sendcount, Datatype sendtype,
                    Object recvbuf, int recvoffset, int recvcount, Datatype recvtype,
                    int root) throws MPJException {...}

public void scatterv(Object sendbuf, int sendoffset, int [] sendcount, int [] displs,
                     Datatype sendtype,
                     Object recvbuf, int recvoffset, int recvcount, Datatype recvtype,
                     int root) throws MPJException {...}
```

Processor's name



References

1. MPI: A Message-Passing Interface Standard - Version 3.1 - 2015